

Análisis y Diseño con Patrones

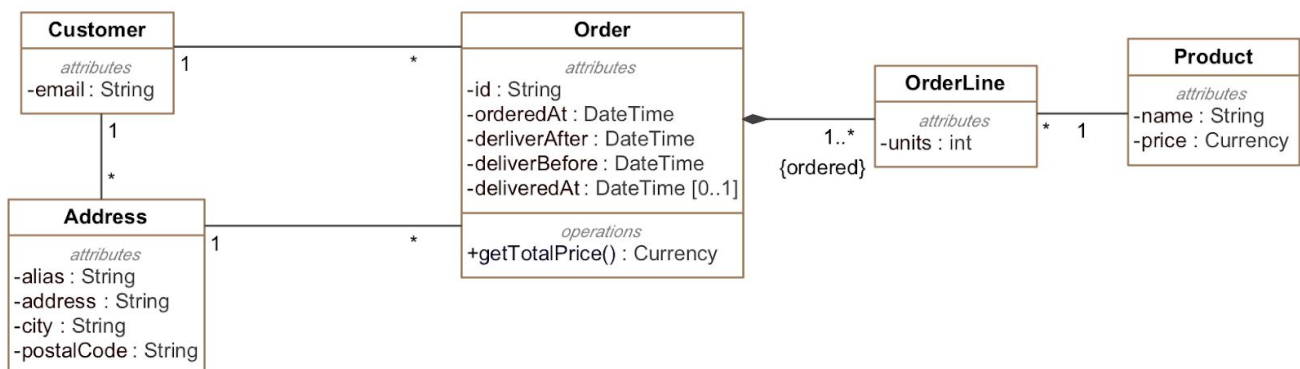
Práctica 1: Principios de diseño, patrones de análisis y arquitectónicos

Nos han contratado para desarrollar un software para gestionar pedidos de compras online. El software formará parte de un sistema mayor, pero sólo tendrá que gestionar información sobre los productos que forman cada pedido, el precio de los pedidos, la fecha y hora en que se hizo el pedido, qué cliente lo hizo, qué día se tiene que entregar, en qué intervalo de horas, y en qué fecha y hora se ha entregado finalmente, si es que ya se ha entregado.

Nota:

- Con la excusa de que "el software formará parte de un sistema mayor" obviaremos muchos elementos que un sistema real tendría. La razón de ello no es otra que la de hacer una práctica didáctica y de la dificultad y dedicación prevista deseadas.

Disponemos ya de un análisis previo que podemos utilizar como punto de partida, pero que habrá que corregir y mejorar:



Notas:

- *Customer* son los clientes; *Address*, las direcciones de entrega de los pedidos; *Order* y *OrderLine* son los pedidos y sus líneas de pedido; y *Product* son los productos. En

OrderLine, *units* es el número de unidades del producto que se piden (por ejemplo, 3 unidades del producto P1).

- Suponed que *Currency* es un tipo de datos para representar cantidades de dinero y que ofrece operaciones como la suma, la resta y la multiplicación sin pérdida de precisión.

Claves de las clases:

- *Customer*: *email*
- *addresses*: *customer* + *alias* (no puede haber dos direcciones de un mismo cliente con el mismo *alias*)
- *Order*: *id*
- *OrderLine*: Orden dentro del pedido
- *Product*: *name*

Restricciones de integridad:

- Para todo pedido, la fecha y hora de inicio de la ventana de entrega (*deliverAfter*) debe ser anterior a la de final (*deliverBefore*) y entre ellas no puede haber menos de 2 horas de diferencia ni más de 6
- Para todo pedido, la fecha y hora de inicio de la ventana de entrega debe ser como mínimo dos horas posterior a la fecha y hora de creación del pedido.
- Para todo pedido, en caso de que la tenga, la fecha y hora de entrega (*deliveredAt*) debe ser posterior a la fecha y hora de creación del pedido

Ejercicio 1 (20%)

Como podemos ver en el diagrama de partida, se ha decidido que el precio total de un pedido sea una información que se calcula cada vez que sea necesario mediante la operación *getTotalPrice*. Pero posteriormente nos damos cuenta de que si un producto cambia de precio, necesitamos que los pedidos ya realizados antes de ese cambio, no cambien sus precios totales.

Nos proponemos buscar una solución que nos permita conocer el precio de cada producto en el momento en que se hizo el pedido así como en cualquier momento pasado. Quisiéramos, pues, que en lugar de un sencillo atributo para el precio, de cada producto pudiéramos saber el precio en cualquier momento del pasado.

- Describe brevemente cómo solucionarías este problema. En caso de que propongas la aplicación de un patrón, explica cuál y razona su conveniencia.
- Propón una solución detallada en forma de diagrama de clases. Muestra sólo las clases que cambien respecto al diagrama del enunciado.

Solución:

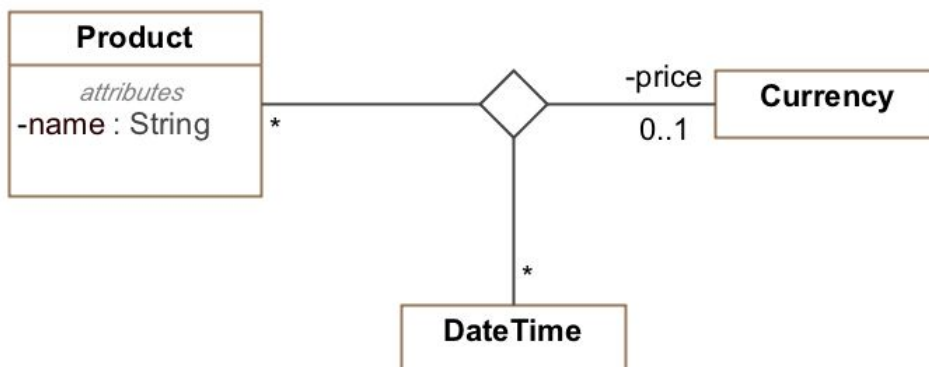
a)

Ahora mismo el precio de un producto es un atributo del mismo y, por tanto, de cada producto sólo sabemos el precio actual. A pesar de que se trata de un atributo y no de una asociación, el patrón que nos permite resolver esta problemática es el patrón Asociación Histórica.

Este patrón nos permite transformar una asociación (o un atributo, como es el caso) en un histórico de los valores que ha tomado la asociación o atributo a lo largo del tiempo. Este es exactamente el problema que queremos resolver.

b)

Como lo que tenemos es un atributo, necesitamos aplicar la variación en que primero extraemos el atributo como una asociación. El resultado final es:



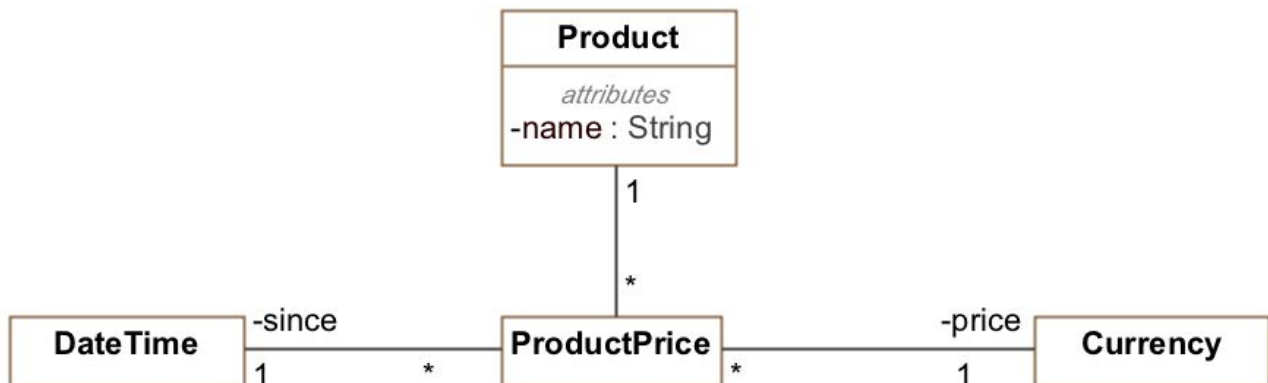
Este diagrama refleja una restricción que antes no teníamos:

- Dado un producto y una fecha-hora concreta, sólo podemos tener un precio. Es decir, un producto sólo puede tener un cambio de precio en un instante (fecha-hora) dado. No resulta de mucho valor, pero sería raro modelar un "*" allí, porque realmente no tiene sentido que en un mismo instante un producto pase a tener dos precios diferentes.

En esta solución asumimos que el precio de un producto es el actual hasta que aparece una asociación con una DateTime posterior, momento en que aquel precio pasa a ser histórico y ya no es el actual. Por lo tanto:

- Estamos ya reflejando que un producto no puede tener dos precios al mismo tiempo, ya que siempre asumimos que si hay dos asociaciones cada una representa un precio que deja de ser válido cuando comienza el siguiente
- Estamos asumiendo que un producto NO puede tener períodos de tiempo en que no tiene ningún precio. Si esta suposición no fuera válida, habría que añadir una asociación (o atributo) que indique el momento en que el precio deja de ser de aplicación y, por tanto, las restricciones correspondientes que indiquen que un producto no puede tener dos precios vigentes en ningún momento.

Por otra parte, muchos usuarios de UML prefieren (preferimos) no utilizar asociaciones ternarias. Se puede hacer modelando la asociación histórica como si tuviera una clase asociativa y luego haciendo una pequeña transformación:



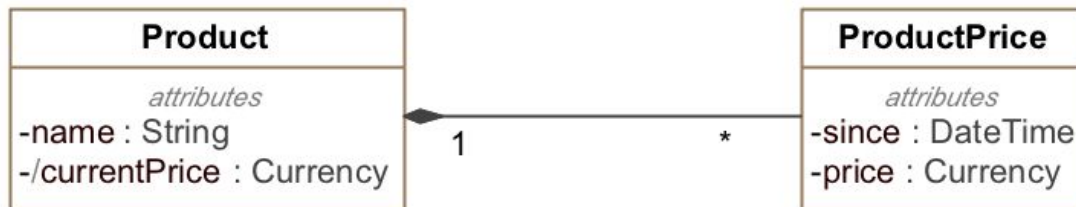
De hecho, como que *DateTime* y *Currency* no son clases sino tipo de datos, que normalmente pondríamos como atributos, esta solución la podemos transformar en :



Por último, podemos ver que *ProductPrice* es un componente de producto, ya que pertenece a un y sólo un producto, no tiene sentido que lo cambiemos de producto y si destruimos un producto tendremos que destruir su histórico de precios. Por lo tanto, podemos modelarlo como:



Por otro lado, parece interesante incluir un atributo que represente el precio que teníamos antes de incluir la asociación histórica; es decir, el precio actual. Este se puede considerar un atributo derivado, ya que se puede derivar del histórico de precios del producto:



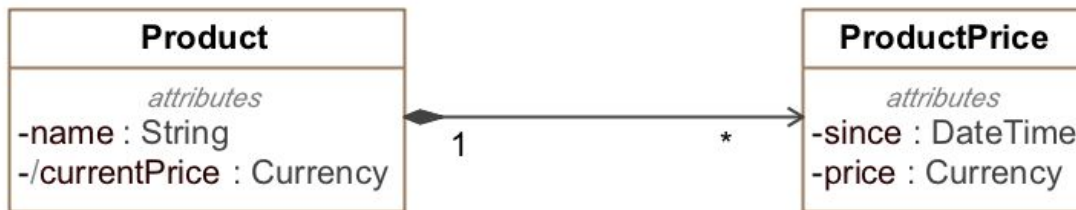
Finalmente puede ser interesante pensar qué representación tendrían estas soluciones a bajo nivel. En realidad, sorprendentemente (o no) todas terminan representando el mismo diseño (que aquí expresamos en Java sin incluir detalles sobre los constructores de las clases):

```

public class Product{
    private List<ProductPrice> priceHistory;
    public Currency getCurrentPrice(){
        if(priceHistory.isEmpty()) return null;
        return priceHistory.last();
    }
}
public class ProductPrice{
    private DateTime since;
    private Currency price;
}
  
```

Como podemos ver, hemos decidido que la asociación entre *Product* y *ProductPrice* se puede navegar de *Product* a *ProductPrice* pero no al revés; es decir, que dada una instancia de *ProductPrice* no podemos obtener a qué instancia de *Product* está asociada, ya que parece que no será necesario.

Esta decisión la hubiéramos podido reflejar también el diagrama UML:



También hemos decidido, si nos fijamos en las versiones en que la fecha *since* y el precio *price* estaban asociados a *ProductPrice*, que sólo son navegables desde *ProductPrice*; por ejemplo, que dada una fecha-hora no podemos saber qué instancias de *ProductPrice* tiene asociadas. En los diagramas donde aparecen estas asociaciones también podríamos haber añadido las flechas de navegabilidad de las asociaciones para reflejarlo.

Ejercicio 2 (20%)

Supongamos que cuando un cliente hace un pedido, elige una ventana de entrega de entre dos y seis horas: es decir, indica de qué a qué hora el cliente estará en la dirección de entrega para a que le lleven lo que ha comprado. Supongamos, también, que tenemos que hacer varios cálculos sobre esta ventana:

- Cuando (en una parte del sistema que no viene al caso) se planifica quién hará la entrega y qué ruta seguirá, hay que poder determinar si una fecha y hora de entrega estimada está dentro o no de la ventana deseada por el cliente y, en caso de que no lo esté, como de alejada está.
- Cuando un pedido es finalmente entregado hay que determinar si la fecha y hora de entrega estaba dentro de la ventana de entrega deseada y, en caso de que no sea así, cuánto tiempo antes o después se ha entregado el pedido.

- a) Cómo modificarías el diagrama de clases para dar respuesta a esta necesidad? En concreto, aplicarías algún patrón de los que hemos visto a este caso? En caso afirmativo, indica cuál. Justifica tu respuesta.

- b) Propón una solución detallada en forma de diagrama de clases. Muestra sólo las clases que cambien respecto al diagrama del enunciado y sólo con los atributos relevantes a este ejercicio.

Solución:

a)

En este caso hay muchas soluciones posibles. La clase Order tiene toda la información necesaria para poder hacer estos cálculos y podríamos, sencillamente, añadir las operaciones necesarias, tales como:

```
isInDeliveryWindow (dateTime: DateTime): Boolean
```

```
secondsOutOfDeliveryWindow (dateTime: DateTime): Int
```

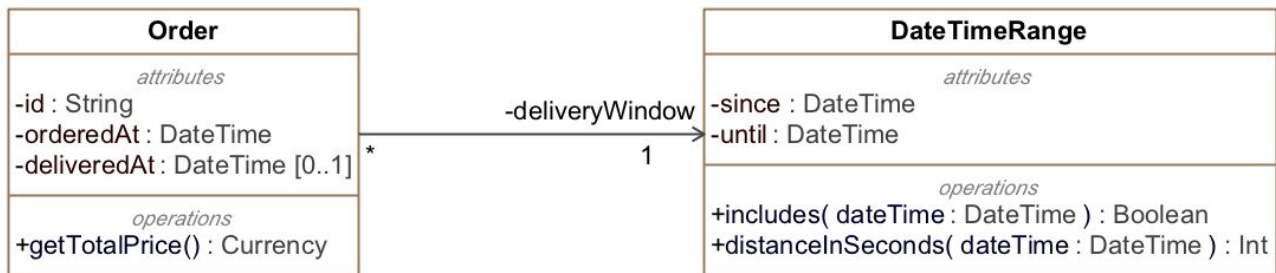
La idea de la segunda operación sería que devolviera cuántos segundos fuera de la ventana de entrega está una fecha hora (y retornara, por tanto, 0, si la fecha hora está dentro de la ventana de entrega).

El inconveniente de esta solución es que este tipo de responsabilidades aparecen en cualquier clase que tenga un par de atributos que representan un rango de valores.

El patrón que nos ayudaría es Rango, ya que permite representar con una clase un rango de valores, y que esta clase represente la semántica de los rangos en general.

b)

Como el patrón indica, creamos una clase que representa un rango de fecha hora y ponemos en la misma la semántica propia de rangos.



En esta ocasión hemos reflejado directamente en el diagrama que *Order* conocerá su ventana de entrega (tendrá asociada una instancia de *DateTimeRange* y podremos navegar a ella) pero que desde *DateTimeRange* no podremos obtener qué pedidos (instancias de *Order*) tenemos asociados.

Ejercicio 3 (20%)

Nota: Para realizar este ejercicio parte del diagrama propuesto en este enunciado y NO tengas en cuenta las modificaciones que has propuesto en el ejercicio 1.

Para el cálculo del precio del pedido disponemos del siguiente pseudocódigo (estilo Java):

```

class Order{

    private List<OrderLine> lines;

    public Currency getTotalPrice() {
        Currency result = Currency.zero;
        for(OrderLine ol : lines) {
            Product p = ol.getProduct();
            // suposem que podem operar sobre Currency (el resultat de p.getPrice)
            // com si es tractés d'un número:
            result.sum(p.getPrice * ol.getUnits());
        }
        return result;
    }
}
    
```

```
}  
  
class OrderLine {  
    private Product product;  
    private int units;  
    public Product getProduct(){  
        return product;  
    }  
    public int getUnits(){  
        return units;  
    }  
}  
  
class Product{  
    private Currency price;  
    public Currency getPrice(){  
        return price;  
    }  
}
```

De este diseño nos preocupa que sea un poco frágil:

- Si, más adelante, ampliamos el sistema para admitir cantidades no enteras en las líneas de pedido, habrá que cambiar el código de la clase *Order*, ya que actualmente se hace el cálculo de multiplicar el precio del producto por la cantidad de la línea.
 - Si, más adelante, aplicamos una solución al problema mencionado en el ejercicio 1, también habrá que cambiar el código de la clase *Order*, ya que es el pedido, la que pide el precio a los productos.
- a) Indica si este diseño satisface o no la Ley de Demeter y razona tu respuesta.
- b) En caso de que no se satisfaga, propón una solución alternativa (también en pseudocódigo del mismo estilo) que sí la satisfaga. En caso de que sí que se satisfaga, indica cómo esto ayuda a cumplir los principios de diseño.

Solución:

- a) No, esta solución no satisface la Ley de Demeter, ya que la clase *Order*, en el método *getTotalPrice*, utiliza la clase *Product*, que no tiene directamente asociada; es decir, obtiene instancias de *Product* que no tiene asociadas, invoca métodos de las mismas y, por tanto, *habla con desconocidos*.
- b) Solución que satisface la Ley de Demeter:

```
class Order{

    private List<OrderLine> lines;

    public Currency getTotalPrice() {
        Currency result = Currency.zero;
        for(OrderLine ol : lines) {
            result.sum(ol.getPrice());
        }
        return result;
    }
}

class OrderLine {
    private Product product;
    private int units;
    public Currency getPrice(){
        return product.getPrice() * units;
    }
}

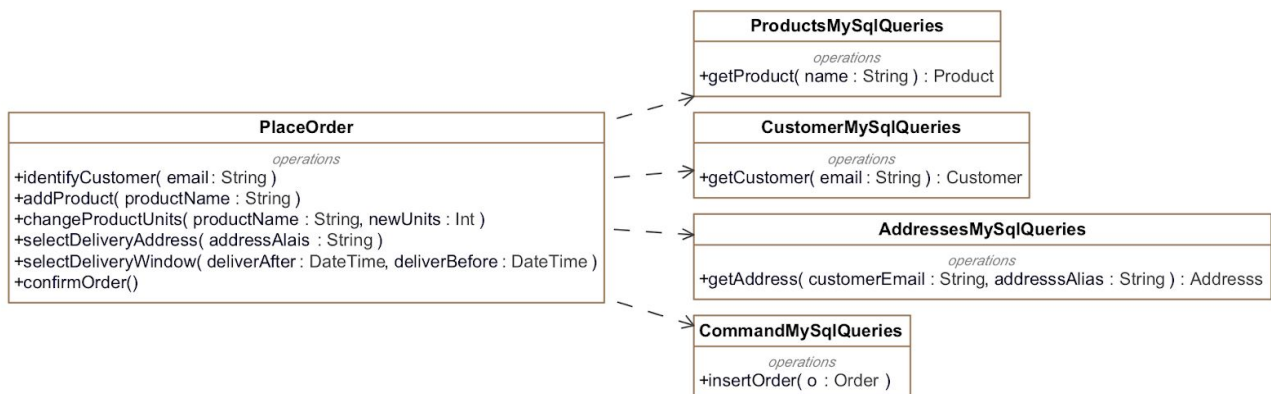
class Product{
    private Currency price;
    public Currency getPrice(){
        return price;
    }
}
```

```
}  
}
```

Ejercicio 4 (20%)

Empezamos ahora a diseñar la arquitectura del software del sistema y hemos decidido utilizar una arquitectura en capas.

Durante el diseño de los primeros elementos nos encontramos en un punto en que disponemos de una clase *PlaceOrder* que nos permite ir dando respuesta, a nivel de abstracción de la lógica, al caso de uso por el que un usuario va añadiendo productos a una cesta y luego hace el pedido:



A medida que el cliente va seleccionando las diversas opciones se van ejecutando los métodos de *PlaceOrder*:

- *identifyCustomer*: Invocada por un subsistema de seguridad (que no viene al caso) informa de cuál es el cliente que está utilizando el sistema. La clase *PlaceOrder* puede buscar los datos del cliente utilizando *CustomerMySQLQueries*.
- *addProduct*: Añadir un producto (1 unidad del mismo) al pedido en preparación. La clase *PlaceOrder* puede buscar los datos del producto utilizando *ProductsMySQLQueries*.
- *changeProductUnits*: Cambia el número de unidades de un producto del pedido en preparación (por ejemplo, para añadir o quitar unidades).

- *selectDeliveryAddress*: Elige una de las direcciones del cliente como dirección de envío. La clase *PlaceOrder* puede buscar los datos de la dirección de entrega utilizando *AddressesMySQLQueries*.
- *selectDeliveryWindow*: Elige la franja horaria de entrega.
- *confirmOrder*: Crea el pedido con todos los datos recogidos y lo registra en la base de datos mediante *CommandMySQLQueries*.

Veamos un fragmento del diseño en pseudocódigo:

```
class PlaceOrder {
    private List<OrderLine> orderLines;
    ...
    public void addProduct(String productName){
        // No sabemos de donde obtener la instancia de ProductsMySQLQuerye que
        // necesitamos:
        ProductsMySQLQueries queries = ???;
        Product p = queries.getProduct(productName);
        orderLines.add(new OrderLine(p, 1));
    }
    ...
}
```

Creas que esta clase cumple todos los principios de diseño que hemos visto? En caso afirmativo, justifica por qué crees que es así. En caso negativo, indica qué principios crees que no cumple e indica qué cambios introducirías en el diagrama de clases para cumplirlos.

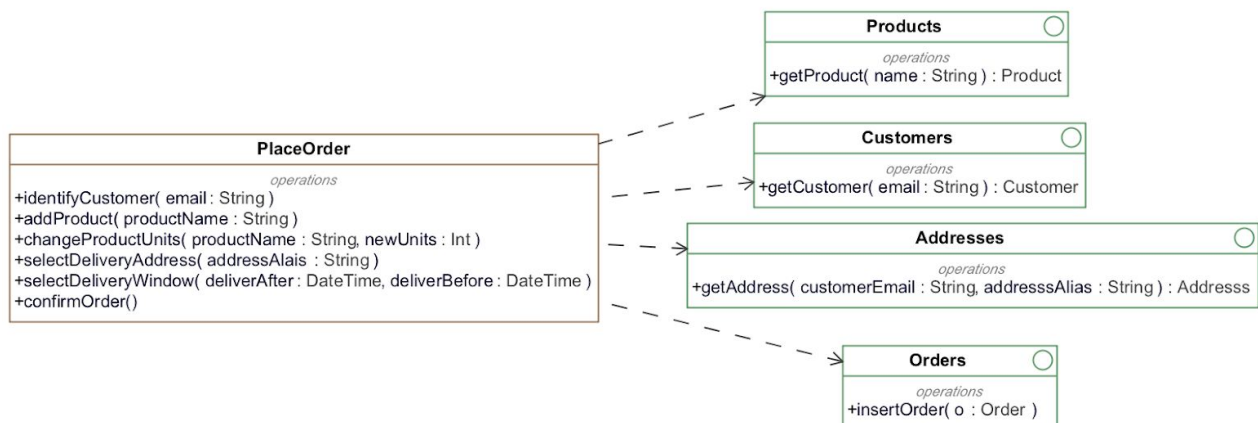
Nota:

- NO es necesario que des solución para el pseudocódigo y, en particular, no es necesario que resuelvas de donde saldría la instancia de *ProductsMySQLQueries* (o equivalente en tu solución).

Solución:

Esta solución se puede cuestionar si satisface más o menos algunos de los principios de diseño. Pero un principio que incumple claramente es el principio de Inversión de dependencias, ya que *PlaceOrder*, que está al nivel de abstracción de la lógica del dominio, depende de clases que tienen un nivel de abstracción mucho más bajo, ya que tienen el nivel de abstracción de la infraestructura; de hecho, el de un fabricante concreto de bases de datos relacionales.

A nivel de diagrama de clases la solución consiste en hacer que *PlaceOrder* dependa de abstracciones y que los detalles también dependan de estas abstracciones. Una forma de lograrlo es crear una abstracción para cada clase demasiado concreta:

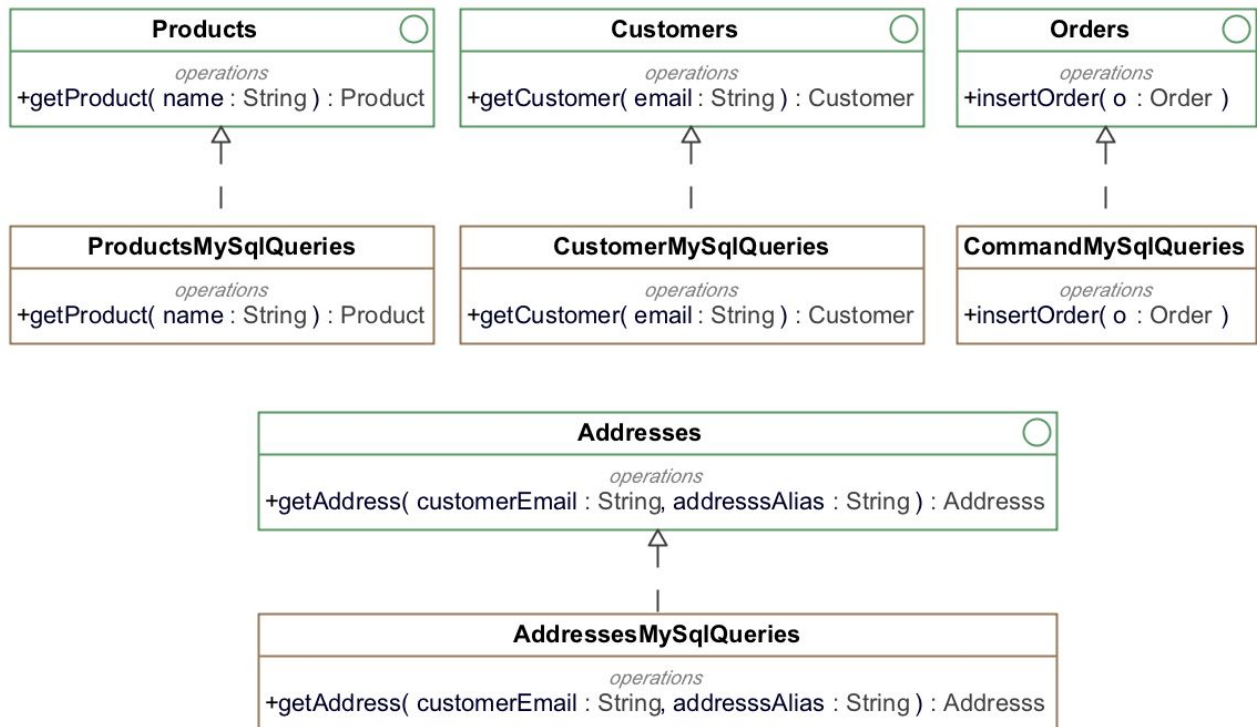


En este caso hemos creado 4 interfaces que representan, al nivel de abstracción del dominio, el conjunto de productos, de clientes, de direcciones y de pedidos que tenemos en el sistema.

Nota: El nombre de la clase *CommandMySQLQueries* del enunciado incluye un error de traducción, ya que la palabra pedido se traduce como order, no como command. En esta solución dejaremos el nombre de la clase original para evitar confusiones mayores, pero la abstracción que hemos creado la hemos llamado Orders, no Commands.

Ahora *PlaceOrder* no depende de detalles sino de otra abstracción que está a su mismo nivel de abstracción.

Entonces, los detalles, que ya teníamos, pasan a depender de las abstracciones. Una forma de dependencia válida es la de subclase (o implementación de interfaz, en este caso) en que una clase depende de su superclase o interfaz:



Esto no resuelve cómo obtendremos, en *PlaceOrder*, las instancias concretas que implementen estas interfaces. De esto habla el ejercicio 5.

Ejercicio 5 (10%)

En el ejercicio anterior hemos planteado una cuestión de diseño. Pero al ver, a bajo nivel de detalle, el diseño propuesto, hemos visto que no sabíamos de dónde obtener una instancia de *ProductsMySQLQueries*. Podríamos convertir todas las operaciones de esta clase en operaciones con ámbito de clase (estáticas) y usarlas directamente desde *PlaceOrder*. Pero nos preocupa que quizás más adelante queremos soportar también la base de datos PostgreSQL (y poder decidir en

cada instalación si utilizamos una u otra base de datos); en tal caso tendríamos *PlaceOrder* acoplado a la clase *ProductsMySQLQueries* y sería complicado poder decidir la base de datos en el momento de la instalación del software.

Propón una solución que permita desacoplar *PlaceOrder* de las clases específicas de MySQL. Esta solución debe estar en consonancia con la solución que ha propuesto para el ejercicio anterior. La solución debe explicar, además, cómo *PlaceOrder* consigue la instancia de la clase que utilizará para consultar un producto a partir de su nombre, resolviendo así la duda surgida en el pseudocódigo del ejercicio anterior.

Solución:

En la solución del ejercicio anterior, ya hemos modelado que *PlaceOrder* solo use (invoque métodos de) abstracciones, interfaces pertenecientes a la clase de dominio, como *Products*, *Customers*, *Addresses* u *Orders*.

En algún momento, sin embargo, necesitamos obtener una instancia de este tipo. Si *PlaceOrder* instanciara ella misma las instancias que necesita se volvería a acoplar a las clases concretas de la capa de servicios técnicos:

```
Customers customers = new CusomterMySQLQueries();
```

Queremos, pues, escoger la implementación de *Customers* (y demás) a utilizar sin acoblar *PlaceOrder* a esa implementación..Este es exactamente el problema que resuelve el patrón de arquitectura Inyección de Dependencias.

La solución consiste en asociar una instancia de *Products*, una de *Customers*, una de *Addresses* y una de *Orders* a *PlaceOrder*. Bajando al nivel de código, *PlaceOrder* tendrá un atributo para cada una de estas instancias. Entonces, en lugar de inicializar la misma *PlaceOrder* estos atributos, le inyectaremos dichas dependencias desde fuera. Proponemos utilizar la variante en que se inyectan a través del constructor:

```
class PlaceOrder{
    private Products products;
    private Customers customers;
    private Addresses addresses;
```

```
private Orders orders;
public PlaceOrder(Products ps, Customers cs, Addresses as, Orders os){
    this.products = ps;
    this.customers = cs;
    this.addresses = as;
    this.orders = os;
}

public void identifyCustomer(String email){
    Customer c = customers.getCustomer(email);
    ...
}
}
```

Hemos mostrado el diseño resultante y cómo se utilizaría una de las abstracciones desde una de las operaciones de la clase.

Ejercicio 6 (10%)

Al inicio del Ejercicio 4 decíamos que hemos decidido aplicar una arquitectura en capas. Utilizaremos las tres capas clásicas de muchos sistemas de información:

1. Presentación
2. Dominio
3. Servicios técnicos

Queremos reflexionar sobre a qué capa pertenece cada una de las clases que han ido surgiendo en el enunciado (y en la solución que has elaborado hasta ahora):

- Clases del primer diagrama: *Order*, *OrderLine*, *Product*, *Customer* y *Address*
- clases del segundo diagrama (Ejercicio 4): *PlaceOrder* y todas las *XxxxMySQLQueries*
- Posibles clases y / o interfaces introducidas en tu solución de los Ejercicios 4 y 5

- a) Indica qué clases (e interfaces, si las hay) pertenecen a la capa de dominio. Razona tu respuesta.

- b) Indica qué clases (e interfaces, si las hay) pertenecen a la capa de servicios técnicos. Razona tu respuesta.

Solución:

a)

Las clases de la capa de dominio trabajan al nivel de abstracción del dominio del problema que nuestro sistema resuelve.

Esto incluye, naturalmente, las clases las instancias de las cuales representan objetos, documentos o valores del mundo real, como un pedido, un producto o un cliente:

- *Order, OrderLine, Product, Customer, Address* y los tipos de los atributos al primer diagrama

También incluye las clases que ofrecen operaciones del nivel del dominio para ser usadas por la capa de presentación, que podemos llamar controladores de dominio:

- *PlaceOrder*

Finalmente, por inversión de dependencias, hemos añadido clases que representan el conjunto de objetos de dominio que el sistema conoce y que eventualmente serán implementadas por la capa de servicios técnicos para ofrecer persistencia:

- *Products, Customers, Addresses y Orders* (de la solución del ejercicio 4)

b)

La capa de servicios técnicos tiene el nivel de abstracción de la infraestructura sobre la que corre nuestro sistema (ficheros, puertos, etc) o el nivel de abstracción de servicios externos como los de las bases de datos. En nuestro caso, por tanto, incluye aquellas clases que implementan el acceso a una base de datos relacional para recuperar datos persistentes:

- *ProductsMySQLQueries, CustomersMySQLQueries, CommandsMySQLQueries, AddressesMySQLQueries*